

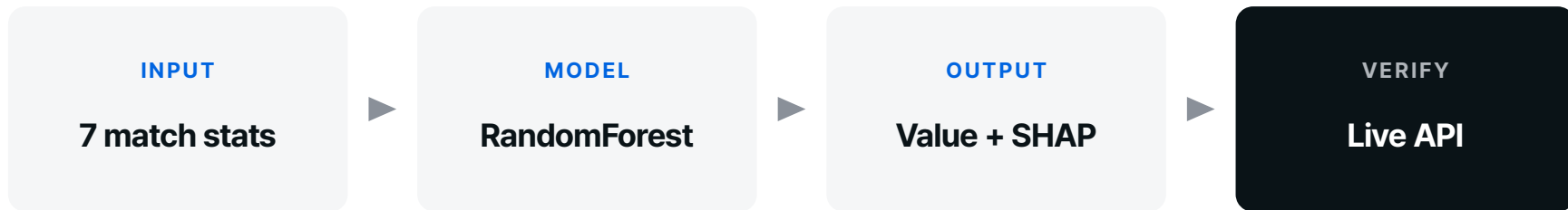
Soccer Player Market Value Prediction.

Predict a player's market value from public performance stats. Then show what drives each number.

Sungjin Choi · STAT 418 · Final Project · Spring 2026

APP 418final-uxdfclcbiixufvv9rpxytej.streamlit.app **API** market-value-api-348858993647.us-central1.run.app

REPO github.com/lacygus/418_final **DOCS** market-value-api-348858993647.us-central1.run.app/docs



PROJECT FRAMING

Predict market value. Explain it. Flag the gaps.

What the project is built to do, and just as important, what it does not claim to do.

PROBLEM

Market value is set by people. It moves slowly and leans on a player's reputation, so it does not always track current on-pitch output.

WHY IT MATTERS

In a transfer, one wrong valuation can cost a club tens of millions. A performance-based estimate gives a second opinion before that decision.

WHAT IT DOES

It predicts a value from public match stats, then attributes the prediction back to the features that pushed it up or down.

WHAT IT IS NOT

It does not try to replace market value. It tests where public stats agree with the market, and where they pull apart.

DATA COLLECTION

Where the data came from

The sources behind the dataset, and the collection problem that ended up shaping the whole project.

SOURCES

Market values, squad lists, and per-season stats from Transfermarkt across 76 clubs. FIFA attributes from Sofifa were tried, then left out of the deployed model.

COLLECTION

Scraped with requests and BeautifulSoup, rate-limited with a browser user-agent. FBref returned a 403, so that source was dropped.

THE FIX

The first scrape took only the top 100 per league. The value range was too narrow and cross-validated R^2 went negative. Full club squads restored the long tail.

TOP FIVE LEAGUES

Premier League**LaLiga****Serie A****Bundesliga****Ligue 1**

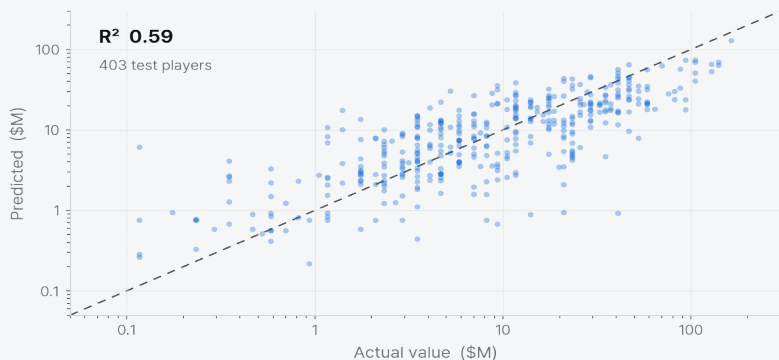
2,013 players · 76 clubs · values ranging \$60K to \$234M (USD)

MODEL & EVALUATION

RandomForest on log value, read with SHAP

Seven raw stats in, a log-scaled value out. Here is how to read the result.

PREDICTED VS ACTUAL · TEST SET



Tight around the diagonal. The model compresses the extremes, so the very high values come in a bit low.

THE 7 FEATURES

Age, appearances, goals, assists, minutes, yellow and red cards. Raw per-season numbers. No position or league yet, the two biggest missing drivers.

PERFORMANCE · TEST SET

R^2 0.59 explains a moderate share of held-out variation

MAE \$9.6M average gap from the real value

WHY SHAP

SHAP splits each prediction into per-feature pushes, so the app can show why a player sits where it does.

SOLUTION ARCHITECTURE

A Streamlit app connected to a Cloud Run API

The model is trained once and saved. The Cloud Run API serves it, and the Streamlit app calls that API to confirm the predictions match.

BUILD

Transfermarkt



Dataset · 2,013



RandomForest



model.pkl

SERVE · RUNTIME

User



Streamlit app

Streamlit Cloud



FastAPI

Cloud Run



model.pkl

TECH STACK



Python



scikit-learn



FastAPI



Streamlit



Docker



Cloud Run



GitHub

APPLICATION & DEMO

Five tabs, and a live parity check

Scout a player and see why it is valued. Then send the same input to the API and confirm the two agree.

WHAT THE APP DOES · FIVE TABS

Scout a player value, 80% interval, SHAP, similar players

Market Movers players the market may be pricing wrong

Leaderboards top players and clubs by league

Model Performance predicted vs actual, residuals

About data sources, exchange rate, limitations

LOCAL vs API PARITY · app-to-API connection

App

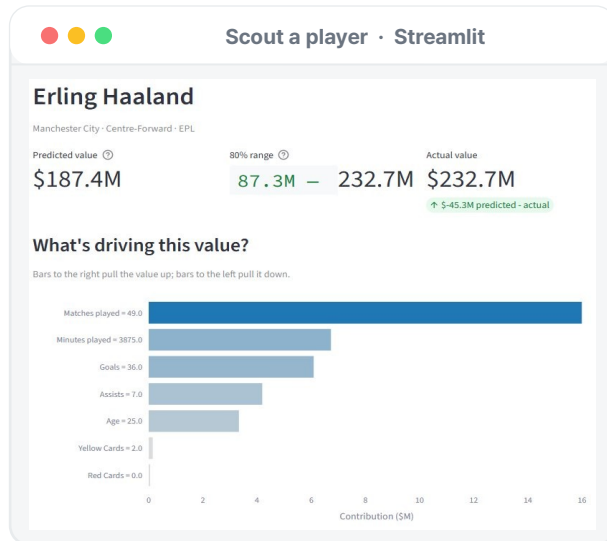
local model.pkl

=

Cloud Run API

same input

Click the button and the app sends the same 7-stat input to the deployed API. **Both return the same prediction.**



CHALLENGES & FUTURE

What broke, what I learned, what's next

The hard parts were data range and deployment. The next steps follow straight from the gaps.

DATA RANGE

The first scrape took only top players, so the value range was too narrow and R^2 went negative. Full squads fixed it.

DEPLOYMENT

Building the container with Podman on Windows broke under WSL. I deployed from source with gcloud, using the same Dockerfile.

AI ASSIST

AI scaffolded the API, validation, and SHAP wiring. The decisions were mine: diagnosing the failed sample, changing the data strategy, writing the limits.

NEXT · 1

Add position and league. My EDA showed these are the biggest missing drivers, so they should lift accuracy the most.

NEXT · 2

Calibrate the interval. The spread across trees is a rough uncertainty proxy, not a real probability yet.

NEXT · 3

Add market signals: contract length, reputation, transfer rumors. These are what real prices lean on.